Jakarta EE

The standard platform for enterprise Java

Servlets · Jakarta Pages (JSP) · CDI · EJB · JPA · REST

Computer Science — Jakarta EE | Lecture 01
Formerly known as **Java EE**

What is Jakarta EE?

Jakarta EE (formerly **Java EE**) is a collection of **open specifications** for building **enterprise applications** in Java.

- It is a **standard**, not a product or a single framework.
- **Vendors implement** the specifications in **application servers**.
- Built on top of **Java SE** (the language + standard library).
- Your components run inside **containers** managed by the server.

> “Enterprise” = web applications, business logic, persistence, messaging and security — built to scale.

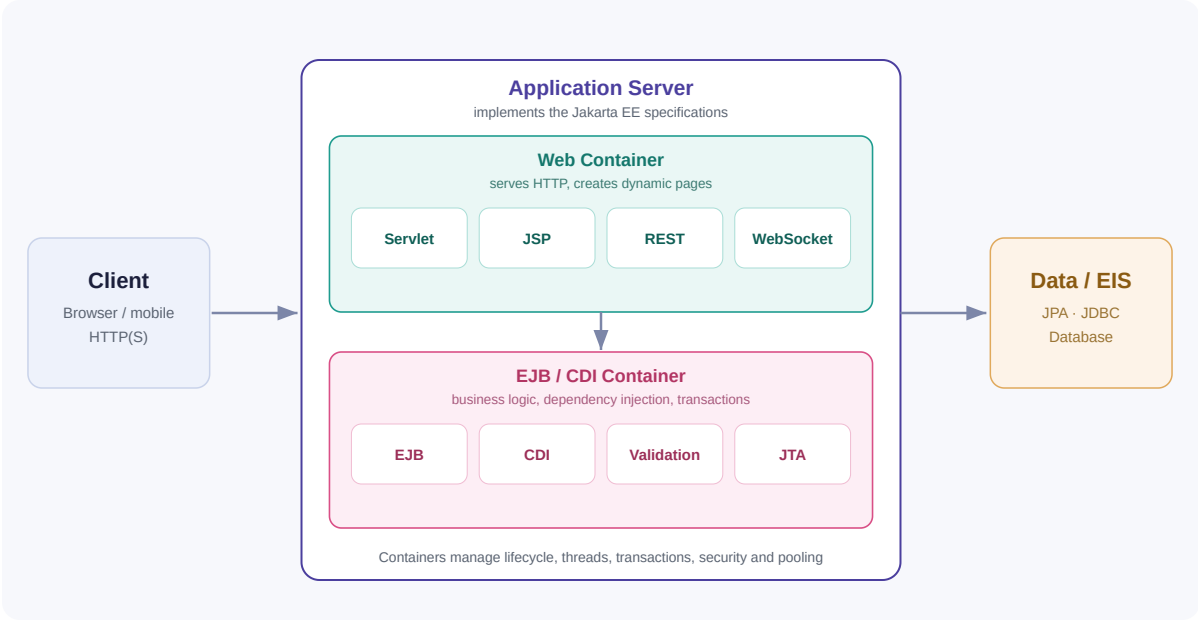
Year	Milestone
1999	J2EE 1.2 — first release (Sun)
2006	Renamed **Java EE 5**
2017	Oracle donates it to Eclipse
2018	Renamed **Jakarta EE**
2025	Current: **Jakarta EE 11**

Since EE 9 the packages use `jakarta.*` instead of `javax.*`.

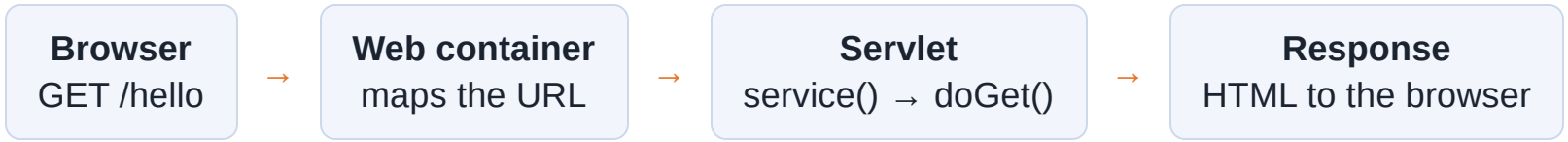
How Jakarta EE Works

- **Specifications** define the contracts (Servlet, JSP, JPA, CDI…).
- A **vendor** implements them: WildFly, Payara, GlassFish, Open Liberty, TomEE.
- You package your application (e.g. a **WAR** or **EAR**) and deploy it.
- The server provides **containers** that host your components and give them services for free.

You focus on **business logic**; the container handles threading, transactions, connection pooling and security.



How a Request Is Handled



- The container receives the **HTTP request** and matches the **URL pattern** to a servlet.
- It calls `service()`, which dispatches to `doGet` or `doPost` based on the HTTP method.
- The servlet writes to a `response` object; the container sends it back to the client.
- **One servlet instance** serves **many requests on many threads** — servlets must be thread-safe.

The same container also serves static files, JSPs and REST endpoints.

Servlets

A **Servlet** is a Java class that runs **inside a web container**, receives HTTP requests and produces HTTP responses.

```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req,
                         HttpServletResponse resp)
            throws IOException {
        resp.setContentType("text/html");
        resp.getWriter().println("Hello, World!");
    }
}
```

Lifecycle — managed by the container

init() — once

↓

service() — every request

↓

destroy() — on shutdown

- Mapped by **annotation** or `web.xml`.
- Read input with `request.getParameter(...)`.
- Write output with `response.getWriter()`.

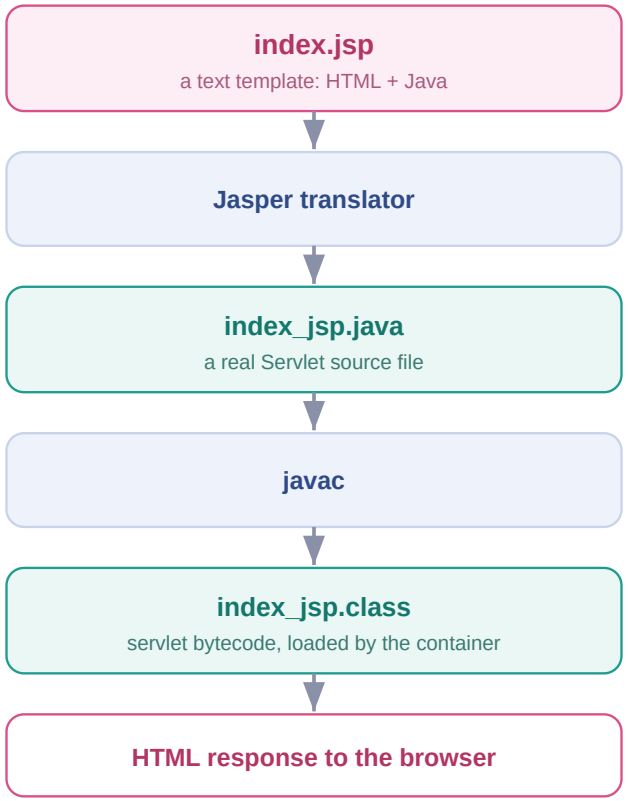
Jakarta Pages (JSP)

A **JSP** is a **text template** that mixes HTML with Java. It is **compiled into a servlet** the first time it is used.

```
<h1>Hello, <%= name %>!</h1>
<p>Rendered at <%= new java.util.Date() %></p>
```

- `<% ... %>` = scriptlet (Java code) · `<%= ... %>` = print an expression.
- **Expression Language** (`${...}`) and **JSTL** keep templates free of Java.
- **MVC:** the servlet is the controller, the JSP is the view.

Same runtime as a servlet — a JSP **is** a servlet, generated for you.

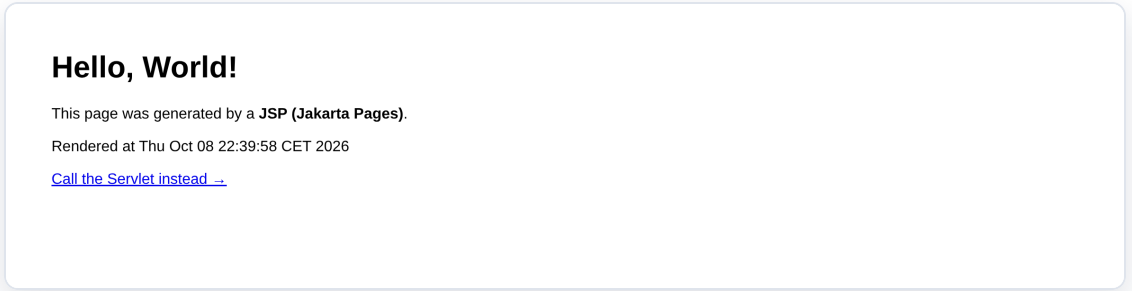


Putting It Together — a Jakarta EE 11 Hello World

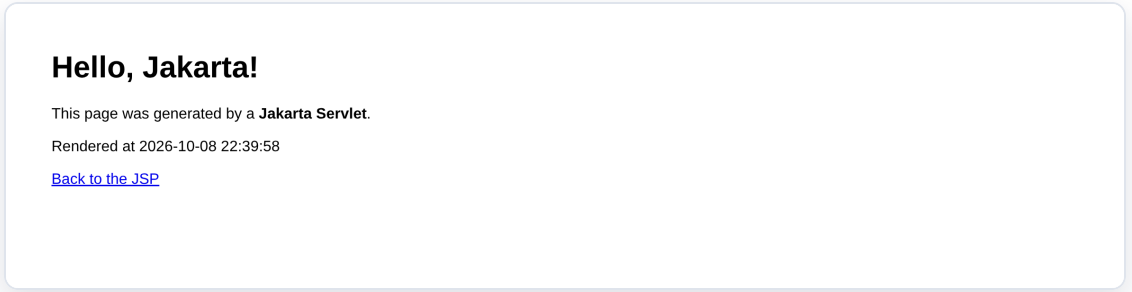
A Maven WAR project — Java 17+, `jakarta.jakartaee-api:11.0.0`

```
hello-world/
├── pom.xml
└── src/main/
    ├── java/com/example/helloworld/
    │   └── HelloServlet.java   (@WebServlet)
    └── webapp/
        ├── index.jsp
        └── WEB-INF/web.xml
```

1. `mvn package` → `target/hello-world.war`
2. Deploy it to a Jakarta EE 11 server
3. Visit `/index.jsp` and `/hello?name=Jakarta`



index.jsp — rendered by the JSP engine



/hello — rendered by the servlet

Both screenshots are the real app running on Apache Tomcat 11 (Servlet 6.1).

The Web Container: Apache Tomcat

Apache Tomcat is the reference implementation of the **Jakarta Servlet** and **Jakarta Pages (JSP)** specifications — a free, open-source **web container**.

Tomcat	Servlet / JSP	Jakarta EE
10.1	6.0 / 3.1	EE 10
11.0	**6.1 / 4.0**	**EE 11**

- Built from **Catalina** (servlet container) + **Jasper** (JSP engine).
- You deploy by dropping a **WAR** into its `webapps/` folder.

> Tomcat is a **web container**, not a full application server: it gives you Servlets, JSP and WebSocket — but **no** EJB, CDI or JPA. Use WildFly / Payara / GlassFish for the full platform.

Download: `tomcat.apache.org` — Tomcat 11 is perfect for our Hello World app.

Installing Tomcat 11 on Windows

1 — Install a JDK 17+
Eclipse Temurin or Oracle JDK. Note the install path.

2 — Download Tomcat 11
From `tomcat.apache.org`, take the **64-bit Windows zip** and extract it to `C:\tomcat11` (avoid spaces in the path).

Shortcut: the **Windows Service Installer (.exe)** on the same page configures `JAVA_HOME` and runs Tomcat as a Windows service for you.

3 — Set `JAVA_HOME` (System → Environment Variables):

```
setx JAVA_HOME "C:\Program Files\Eclipse Adoptium\jdk-21-hotspot"
```

4 — Start Tomcat

```
cd C:\tomcat11\bin
startup.bat
```

5 — Verify — open http://localhost:8080; the Tomcat welcome page should appear.

6 — Deploy the app — copy `hello-world.war` into `C:\tomcat11\webapps\`, then open http://localhost:8080/hello-world/.

```
shutdown.bat
```

Summary & References

Jakarta EE in one line

- A set of **specifications** for enterprise Java.
- Implemented by **application servers**; your code runs in **containers**.
- Current version: **Jakarta EE 11** (Java 17+, `jakarta.*`).

Servlets & JSP

- **Servlet** = Java class that handles HTTP requests.
- **JSP** = HTML template compiled into a servlet.
- Together they build the **web layer**; CDI / EJB / JPA add the rest.

Run the Hello World project

```
cd hello-world
mvn package   # builds target/hello-world.war
```

References & further reading

- Jakarta EE — https://jakarta.ee/
- EE 11 Platform — https://jakarta.ee/specifications/platform/11/
- Servlet 6.1 — https://jakarta.ee/specifications/servlet/6.1/
- Pages (JSP) 4.0 — https://jakarta.ee/specifications/pages/4.0/
- Apache Tomcat 11 — https://tomcat.apache.org/

Next lecture: the servlet lifecycle and request handling in depth.